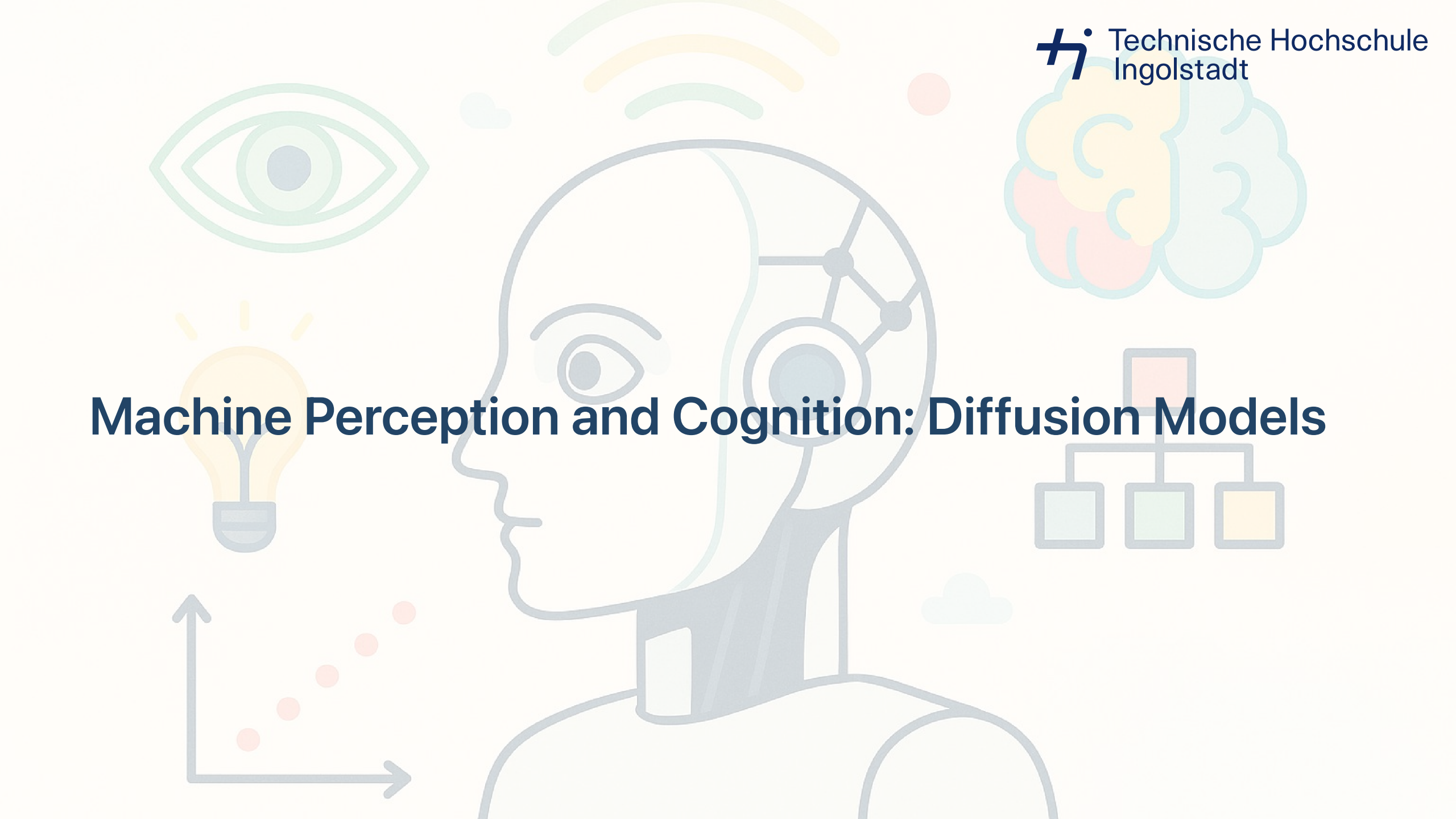
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Diffusion Models

What is a Diffusion Model?

A **diffusion model** is a generative model that learns to **reverse a gradual noising process**.

The core idea:

1. **Forward process**: progressively add Gaussian noise to a clean image $\mathbf{x}_0$ over T steps until it becomes pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2. **Reverse process**: train a neural network to undo one small denoising step at a time — starting from pure noise, recover a clean image

Key properties:

- Training is **stable** compared to GANs (no adversarial game)
- Generated images often have **very high perceptual quality**
- Closely related to **denoising autoencoders**

Intuition: The Forward Process

Imagine taking a clean photograph and adding a tiny amount of Gaussian noise at each step:

$$\mathbf{x}_0 \xrightarrow{+\boldsymbol{\epsilon}_1} \mathbf{x}_1 \xrightarrow{+\boldsymbol{\epsilon}_2} \mathbf{x}_2 \to \cdots \xrightarrow{+\boldsymbol{\epsilon}_T} \mathbf{x}_T$$

After T steps the image looks like pure Gaussian noise — all structure is destroyed.

The forward process is **fixed** (not learned) and defines a schedule of noise levels $\beta_1 < \beta_2 < \cdots < \beta_T$ where $\beta_t \in (0, 1)$ controls how much noise is added at step t.

Key insight: if T is large enough and the noise increments small enough, $\mathbf{x}_T$ is indistinguishable from $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

Forward Process: Mathematical Formulation

The forward process is a **Markov chain** with Gaussian transitions:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t;\ \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t \mathbf{I}\right)$$

At each step the previous sample is slightly scaled towards zero and Gaussian noise is added.

Forward Process: Mathematical Formulation

Defining $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, there is a **closed-form** expression for any arbitrary step t:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_t;\ \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\ (1 - \bar{\alpha}_t)\mathbf{I}\right)$$

This means $\mathbf{x}_t$ can be sampled directly from $\mathbf{x}_0$ **without simulating all intermediate steps**:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Forward Process

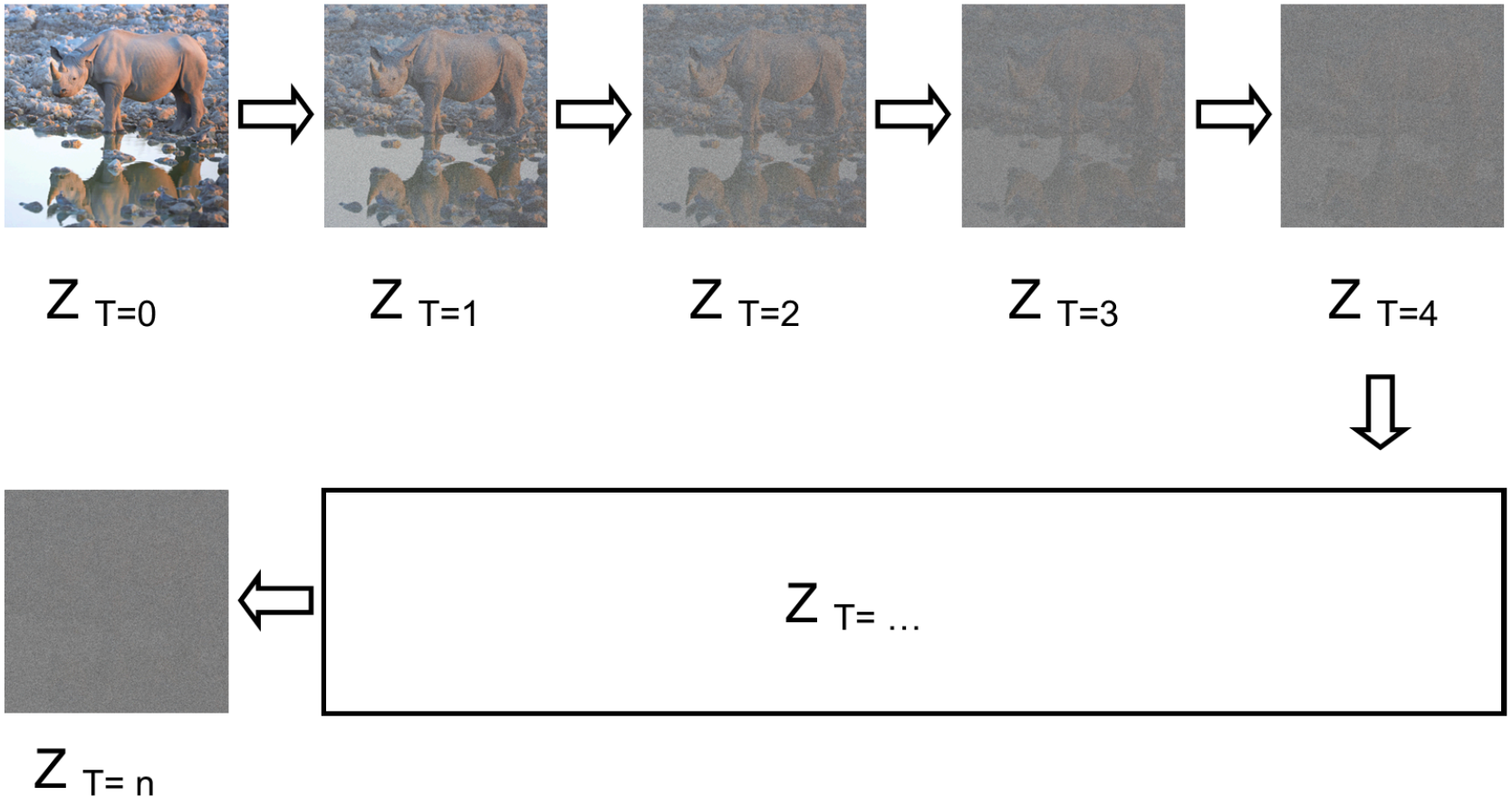


Image source: MrAlanKoh, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Example_of_Forward_Diffusion_Models.png

The Reverse Process

To generate new images we need to **run the process backwards**: start from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iteratively remove noise.

The true reverse transition $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ **without access to** $\mathbf{x}_0$ is intractable — it would require integrating over all possible clean images that could have produced $\mathbf{x}_t$.

The Reverse Process

Key idea: approximate it with a learned Gaussian:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1};\, \boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\, \sigma_t^2 \mathbf{I}\big)$$

where the **mean** $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ is predicted by a neural network parametrized by θ.

The variance σ_t^2 is typically fixed to a schedule (e.g. $\sigma_t^2 = \beta_t$).

Training Objective

After simplification, Ho et al. (2020) show the objective reduces to predicting the noise $\boldsymbol{\epsilon}$ that was added to $\mathbf{x}_0$:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2\right]$$

where:

- $t \sim \mathcal{U}\{1, \dots, T\}$ is sampled uniformly
- $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the true noise added
- $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$ is the noisy image
- $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ is the noise **predicted** by the network

This is a simple **mean squared error** regression task — no adversarial game, no mode collapse.

Training Objective

In stochastic gradient descent, this expectation is optimized with single-sample or mini-batch estimates such as:

$$\ell(\theta; \mathbf{x}_0, t, \boldsymbol{\epsilon}) = \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2$$

Training Algorithm (DDPM)

Training loop:

```
repeat:
  x_0  ← sample from training data
  t    ← sample uniformly from {1, …, T}
  ε    ← sample from N(0, I)
  x_t  = sqrt(ᾱ_t) · x_0 + sqrt(1 − ᾱ_t) · ε    # closed-form forward step
  loss = ‖ε − ε_θ(x_t, t)‖²                       # noise prediction MSE
  update θ by gradient descent on loss
```

Sampling / Inference

We start with pure noise and gradually denoise it:

$$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Then denoise step-by-step (for $t = T, \ldots, 1$):

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \, \mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \text{ for } t > 1$$

Sampling / Inference

- $\mathbf{x}_t$: Noisy image at step t
- $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: Initial, purely noisy image at step T
- $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$: Neural network prediction of noise at step t
- $\alpha_t = 1 - \beta_t$: Signal preservation rate at step t
- β_t: Noise injection rate at step t
- σ_t: Standard deviation of noise added in reverse step
- $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: Stochastic noise injection ($\mathbf{z} = \mathbf{0}$ for $t = 1$)

Sampling / Inference

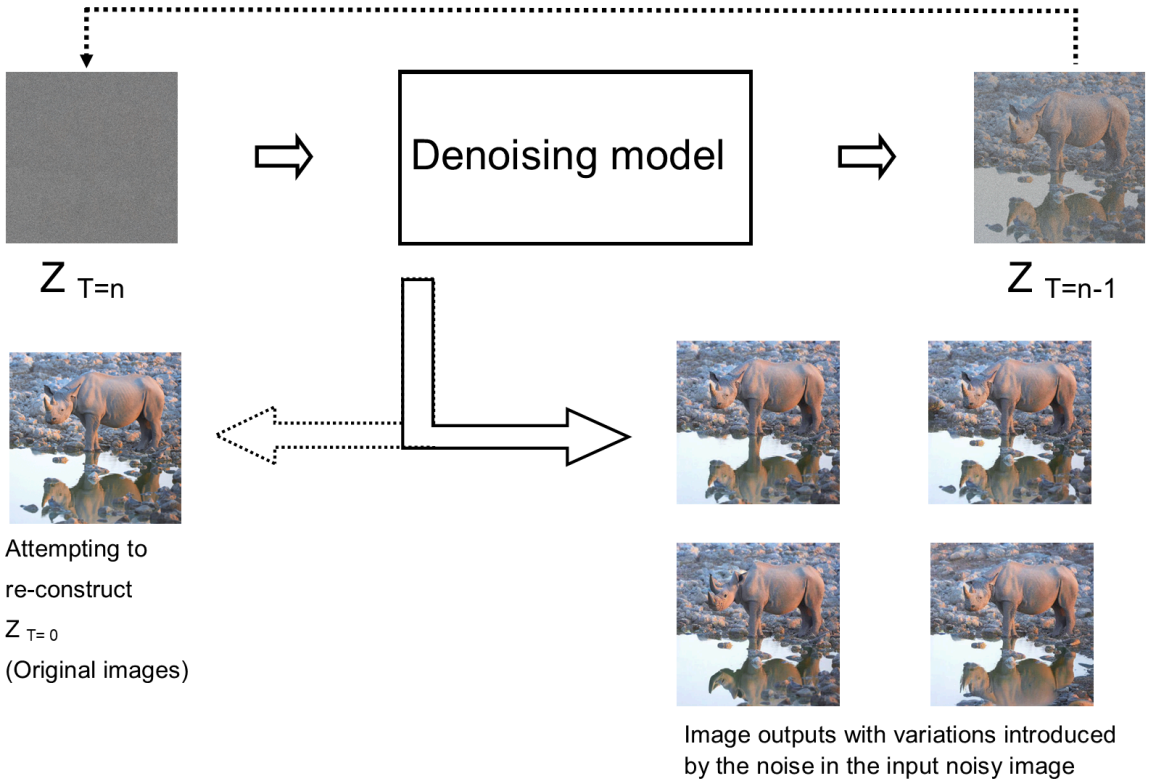


Image source: MrAlanKoh, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Example_of_Denoising_Diffusion_models.png

Connection to Denoising Autoencoder

The diffusion model training objective is **closely related** to the denoising autoencoder:

Aspect	DAE	Diffusion Model
Corruption	Single, fixed noise level	Multiple noise levels $t = 1, \ldots, T$
Target	Reconstruct clean $\mathbf{x}_0$	Predict noise $\boldsymbol{\epsilon}$ (or equivalently $\mathbf{x}_0$)
Conditioning	None	Timestep t
Loss	$\lvert\mathbf{x}_0 - \hat{\mathbf{x}}_0\rvert_2^2$	$\lvert\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\rvert_2^2$

U-Net as the Denoising Network

The denoising network $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ must:

- Take a **noisy image** $\mathbf{x}_t$ (same spatial resolution as the output)
- Take the **timestep** t as a conditioning signal
- Output a **noise map** of the same spatial resolution as the input

U-Net as the Denoising Network

The **U-Net** architecture is a natural fit:

- Its encoder–decoder structure with **skip connections** preserves spatial detail at every scale
- The bottleneck captures global context; skip connections restore fine detail
- The same architecture that excels at **image segmentation** is repurposed here for image denoising

The denoising U-Net used in diffusion models extends the original U-Net with **residual blocks** for more stable gradient flow through many layers.

U-Net Architecture

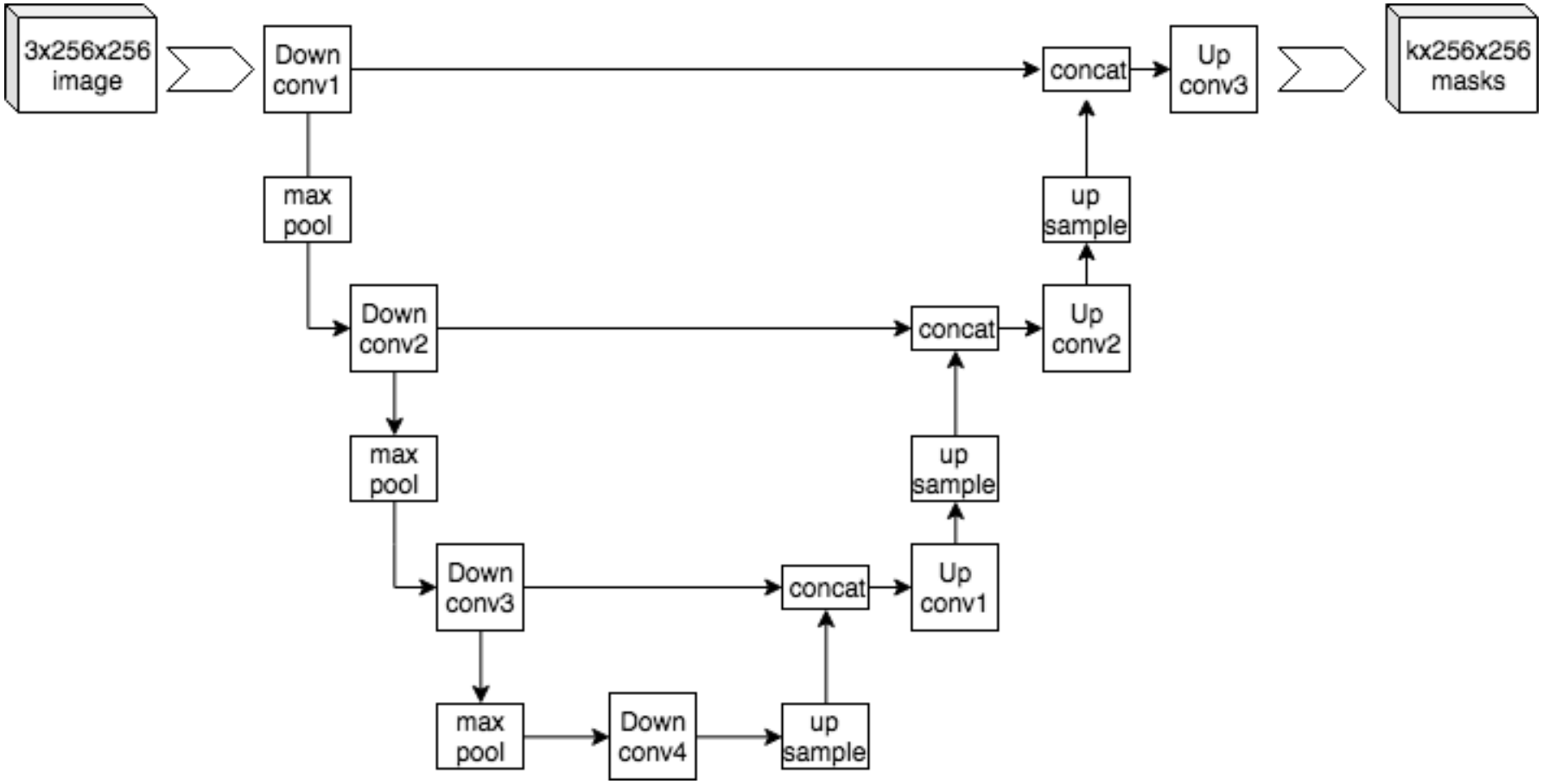


Image source: Mehrdad Yazdani, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Example_architecture_of_U-Net_for_producing_k_256-by-256_image_masks_from_1_256-by-256_RGB_image.png

U-Net Architecture for Diffusion

The U-Net used in DDPM processes the noisy image $\mathbf{x}_t$ through a series of **encoder** and **decoder** blocks connected by skip connections. Schematically, for some total downsampling factor $s > 1$:

$$\underbrace{(H \times W \times C)}_{\mathbf{x}_t} \xrightarrow{\text{Enc}} \underbrace{(H/s \times W/s \times C')}_{\mathbf{h}_{\text{mid}}} \xrightarrow{\text{Dec}} \underbrace{(H \times W \times C)}_{\hat{\boldsymbol{\epsilon}}}$$

U-Net Architecture for Diffusion

Encoder blocks (repeated at each resolution scale):

- GroupNorm → SiLU activation → Conv2D → timestep injection → Conv2D → Downsample

Decoder blocks (symmetric, receive skip from encoder):

- Concatenate skip connection → GroupNorm → SiLU → Conv2D → timestep injection → Conv2D → Upsample

Skip connections pass fine spatial detail from encoder to decoder at each resolution level, ensuring precise localization of denoised features.

SiLU / Swish Activation Function

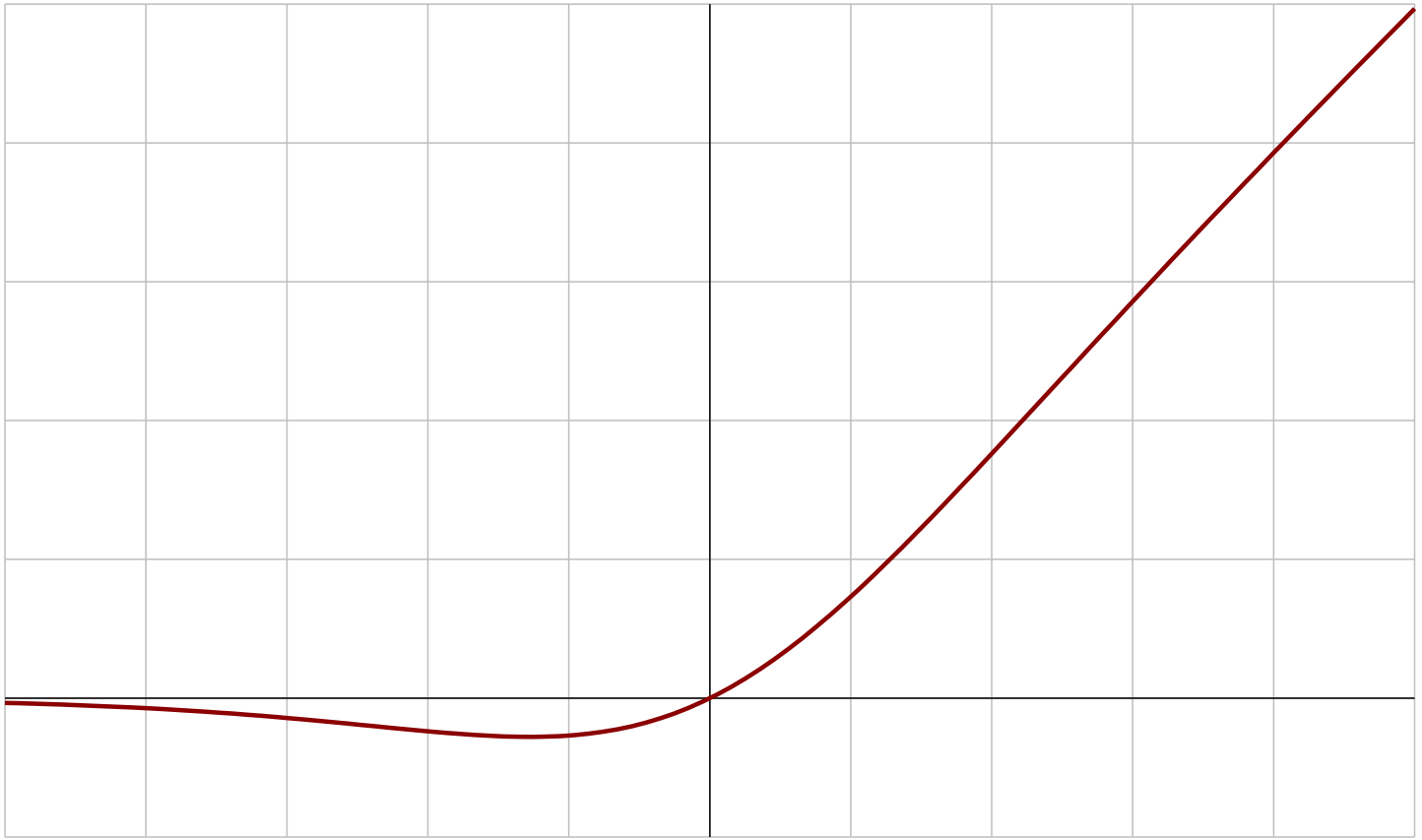


Image source: Romain Mondon-Cancel, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Swish.svg

Timestep Conditioning

The network must know **how much noise** is in $\mathbf{x}_t$ — this is conveyed via the timestep embedding.

Sinusoidal timestep embedding (similar to positional encoding in Transformers, where d is the embedding dimension):

$$\mathrm{emb}(t)_{2i} = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \mathrm{emb}(t)_{2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

The embedding is then projected with two linear layers + SiLU:

$$\mathbf{e}_t = \mathbf{W}_2\,\mathrm{SiLU}(\mathbf{W}_1\,\mathrm{emb}(t))$$

Hint: The 10000 is a fixed scale hyperparameter that spreads sinusoidal frequencies across embedding dimensions.

Timestep Conditioning

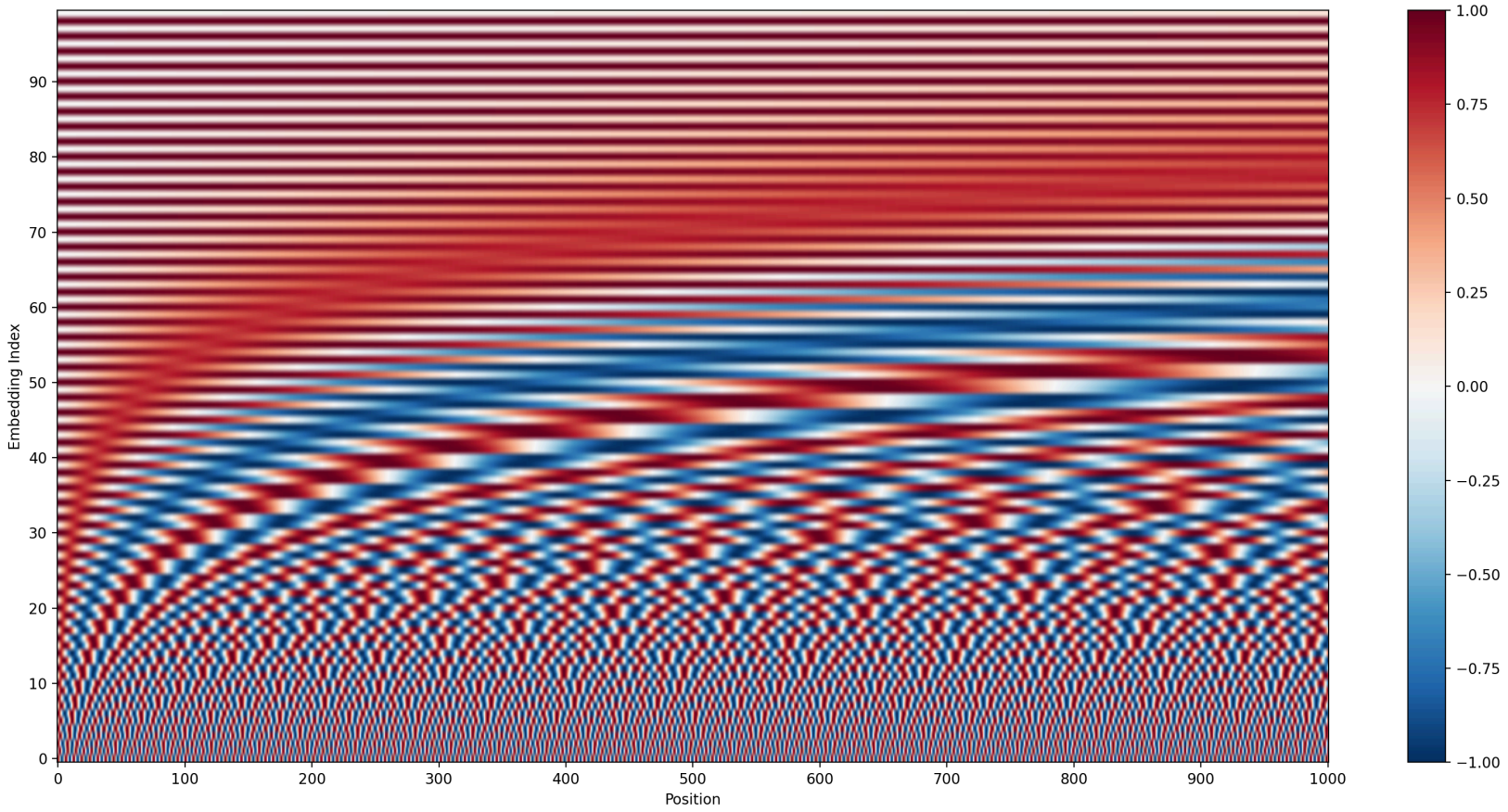


Image source: Nils Blümer, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Absolute_positional_encoding.png

Timestep Conditioning

The timestep vector $\mathbf{e}_t$ is projected to the channel dimension, broadcast across the spatial grid, and then added inside each residual block:

$$\mathbf{h}' = \mathbf{h} + \text{broadcast}(\mathbf{W}_t \mathbf{e}_t)$$

This allows a **single network** to perform denoising at all noise levels $t = 1, \ldots, T$.

Stable Diffusion: Motivation

Problem: Running diffusion in pixel space is computationally expensive.

- For a 512×512 RGB image: each forward pass operates on a tensor of size $512 \times 512 \times 3$
- With $T = 1000$ steps, training touches hundreds of millions of pixel locations per sample, and the actual neural-network FLOP count is much larger
- The bulk of the computation is spent on **low-level perceptual detail** (texture, noise) rather than semantic structure

Stable Diffusion: Motivation

Solution: Instead of denoising in pixel space, apply the diffusion process in the **compressed latent space** of a pre-trained autoencoder.

Benefits:

- Spatial resolution is reduced by a factor of 8 (e.g. $512 \times 512 \to 64 \times 64$)
- High-quality reconstruction is handled by the decoder — the denoising U-Net only needs to model **semantic content**

See Rombach et al. (2022) for the paper.

Stable Diffusion: Architecture Overview

Three main components:

1. **Variational Autoencoder (VAE)**: encodes images into a compact latent space and decodes latents back to pixel space
2. **Denoising U-Net**: performs the diffusion process in latent space, conditioned on a text embedding
3. **Text encoder** (e.g. CLIP): converts a text prompt into a sequence of embeddings used to guide the denoising

The VAE and text encoder are **pre-trained and frozen** during latent diffusion training. Only the U-Net weights are updated.

Contrastive Language-Image Pretraining (CLIP)

From Radford et al. (2021):

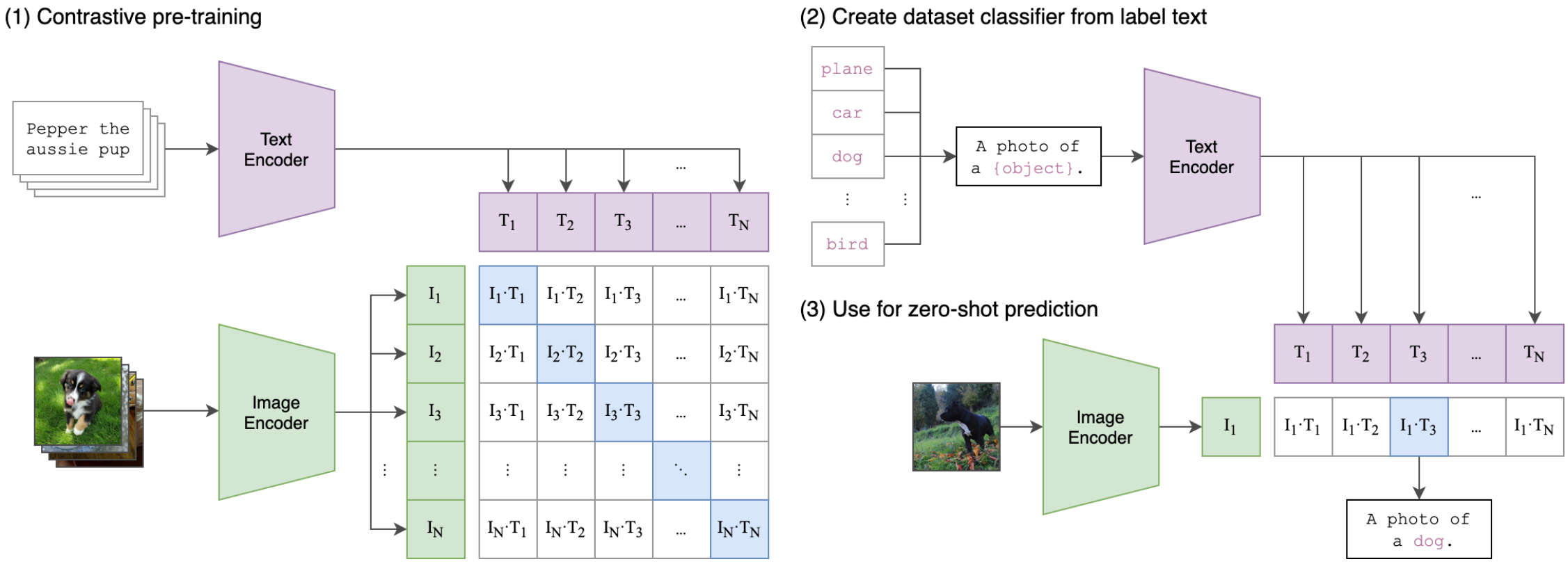


Image source: OpenAI, MIT , via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Contrastive_Language-Image_Pretraining.png

Stable Diffusion: Architecture Overview

Stable Diffusion (Rombach et al. 2022):

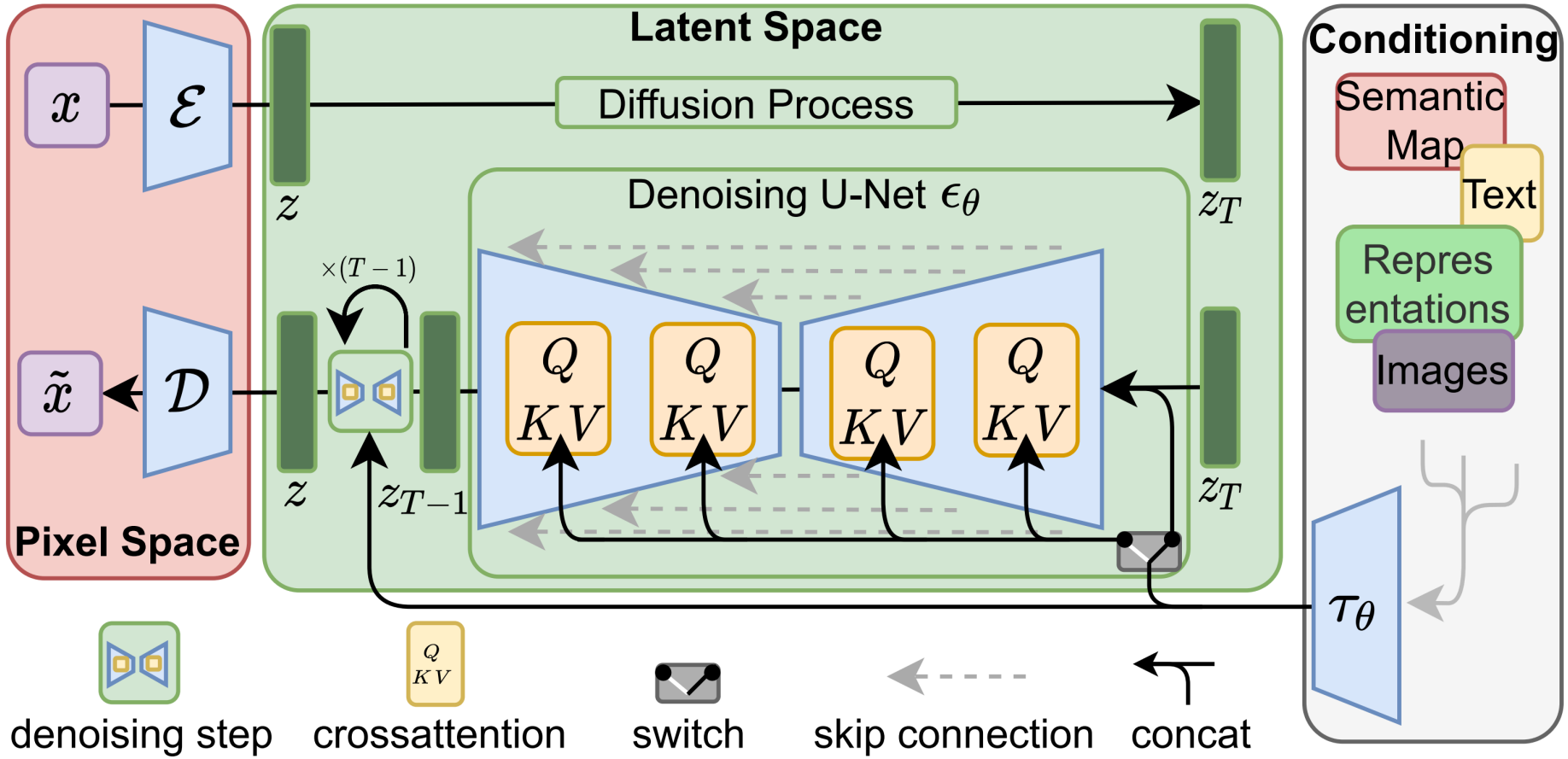


Image source: Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, Björn Ommer, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Diffusion_Architecture.png

Stable Diffusion: VAE

The **VAE** compresses images to a low-dimensional latent representation:

Encoder $\mathcal{E}$: maps pixel image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$ to an approximate posterior over a latent tensor $\mathbf{z} \in \mathbb{R}^{h \times w \times c}$

$$q_{\mathcal{E}}(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\mathbf{z};\, \boldsymbol{\mu}_{\mathcal{E}}(\mathbf{x}), \mathrm{diag}\big(\boldsymbol{\sigma}^2_{\mathcal{E}}(\mathbf{x})\big)\big), \quad h = H/f,\ w = W/f, \quad f \in \{4, 8\}$$

Decoder $\mathcal{D}$: reconstructs pixel image from latent

$$\hat{\mathbf{x}} = \mathcal{D}(\mathbf{z})$$

For Stable Diffusion (Rombach et al. 2022), $f = 8$: a 512×512 image maps to a latent tensor of shape $64 \times 64 \times 4$.

The diffusion forward/reverse process operates entirely on $\mathbf{z}$:

$$\mathbf{z}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \xrightarrow{\text{reverse}} \hat{\mathbf{z}}_0 \xrightarrow{\mathcal{D}} \hat{\mathbf{x}}$$

Text Conditioning with CLIP

To enable **text-to-image** generation, the denoising U-Net is conditioned on a text prompt.

CLIP text encoder (Radford et al. 2021):

- Tokenizes the text prompt into a sequence of tokens
- Passes tokens through a Transformer to produce a sequence of embeddings $\mathbf{c} \in \mathbb{R}^{L \times d}$ (L tokens, each d-dimensional)

What is Cross-Attention?

In **self-attention** (used inside the Transformer), a single sequence attends to itself:

> *"How much should each element look at every other element in the same sequence?"*

Cross-attention generalises this to **two different sequences**:

> *"How much should each element of sequence A look at each element of sequence B?"*

What is Cross-Attention?

Role	Self-Attention	Cross-Attention
Queries $\mathbf{Q}$	Sequence A	Sequence A
Keys $\mathbf{K}$	Sequence A	Sequence B
Values $\mathbf{V}$	Sequence A	Sequence B

This lets one sequence **read information selectively** from another.

Cross-Attention: Intuition

Think of a **painter following a text description**:

- The **painter's canvas** (image features) provides the **queries** — "what am I currently working on?"
- The **text description** (text tokens) provides the **keys and values** — "what information can I offer?"
- At every brushstroke (spatial position), the painter **glances at the most relevant words** in the description to decide what to paint next

The result: each spatial location in the image feature map is updated with a **weighted blend of text information**, where the weights are determined by how relevant each text token is to that location.

Text Conditioning with CLIP

The embeddings $\mathbf{c}$ are injected into attention blocks throughout the U-Net via **cross-attention**:

$$\text{CrossAttn}(\mathbf{h}, \mathbf{c}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

where:

- $\mathbf{Q} = \mathbf{W}_Q\mathbf{h}$ — queries from **image features $\mathbf{h}$**
- $\mathbf{K} = \mathbf{W}_K\mathbf{c}$, $\mathbf{V} = \mathbf{W}_V\mathbf{c}$ — keys and values from **text embeddings $\mathbf{c}$**
- d_k — dimensionality of the queries and keys

This allows every spatial location in the image to **attend to the most relevant parts of the text prompt**.

Stable Diffusion: Inference

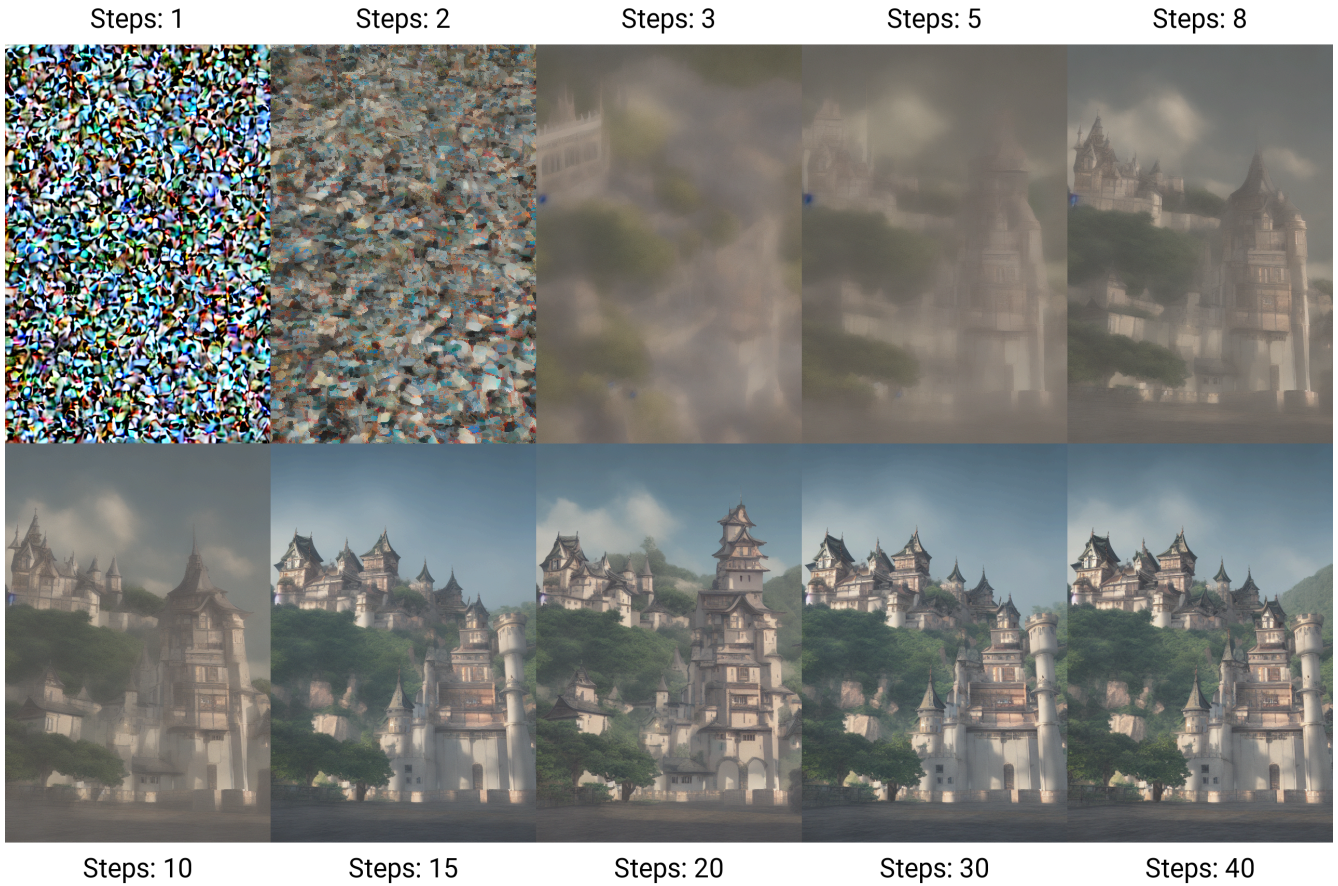


Image source: Benlisquare, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:X-Y_plot_of_algorithmically-generated_AI_art_of_European-style_castle_in_Japan_demonstrating_DDIM_diffusion_steps.png

Diffusion Models vs. VAE vs. GAN

Property	VAE	GAN	Diffusion
Training stability	✅ Stable	❌ Adversarial	✅ Stable
Sample quality	⚠️ Blurry	✅ Sharp	✅ Very sharp
Diversity	✅ Good	❌ Mode collapse	✅ High
Inference speed	✅ Fast (single pass)	✅ Fast (single pass)	❌ Slow (many steps)
Latent space	✅ Structured, smooth	❌ Implicit	⚠️ Usually no compact learned latent in pixel-space DDPMs

Summary: Diffusion Models

- **Forward process**: a fixed Markov chain that gradually adds Gaussian noise to the data over T steps; the noisy image at any step t is $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$
- **Reverse process**: a learned Gaussian Markov chain parametrized by $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ that iteratively denoises pure noise into a data sample
- **Training**: minimizes the **simplified noise-prediction loss** $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2$
- **Sampling**: iterative stochastic denoising over T steps using the learned reverse Gaussian transitions

Summary: U-Net for Diffusion

- The denoising network ϵ_θ uses a **U-Net** with encoder–decoder structure and skip connections — the same spatial resolution in and out makes it ideal for pixel-wise noise prediction
- **Residual blocks** with SiLU activation provide stable gradient flow through many layers
- **Sinusoidal timestep embeddings** are injected into every residual block, enabling a single network to denoise at all T noise levels
- **Cross-attention** injects text or other conditioning signals into attention blocks throughout the U-Net

Summary: Stable Diffusion

- **Latent Diffusion Models** move the diffusion process from pixel space into the **compressed latent space** of a pre-trained VAE — reducing spatial computation by a factor of 16× (for $f = 4$) to 64× (for $f = 8$)
- The **VAE** encodes images to a $64 \times 64 \times 4$ latent (for 512×512 input) and decodes back to pixels; it is frozen during diffusion training
- **CLIP text embeddings** are injected into the U-Net via **cross-attention**, enabling flexible text-to-image generation